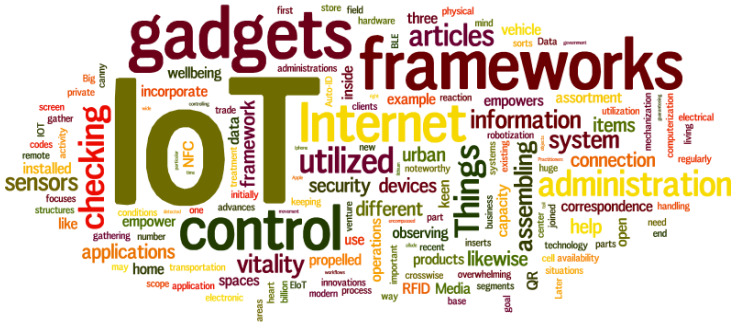
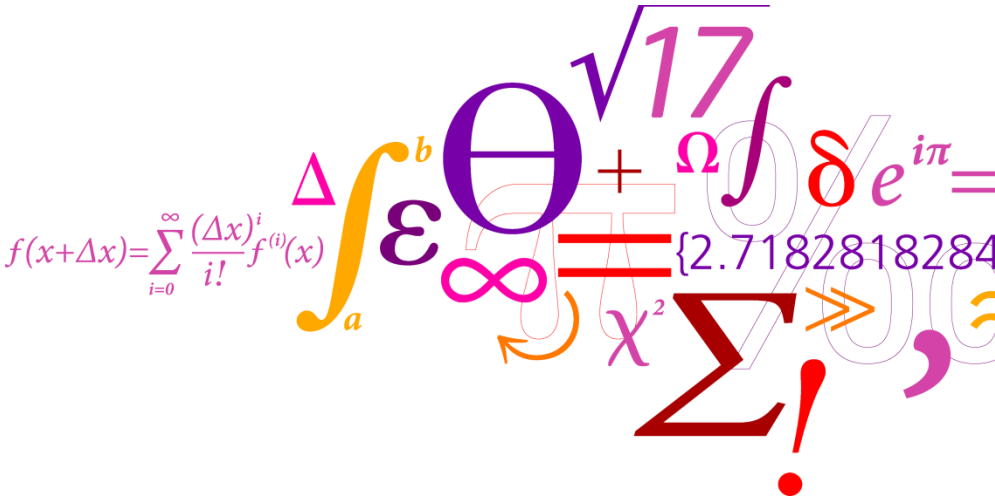


STM32 code structure and Blink

Course 34373

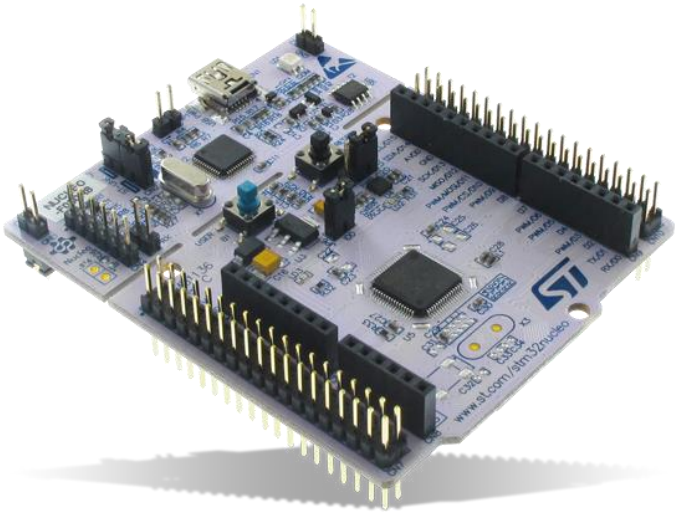
Martin Nordal Petersen
mnpe@fotonik.dtu.dk



DTU Fotonik
Department of Photonics Engineering

Agenda

- Pinout tab and GPIO configuration in CubeMX
- Alternative pin mapping
- Clock configuration
- Example project using CubeMX
- main.c code structure
- Compiling and downloading to Nucleo board
- Altering code to perform *Blink (*MCU version of *Hello World)*
- Additional ST tools (*CubeProgrammer* & *CubeMonitor*)

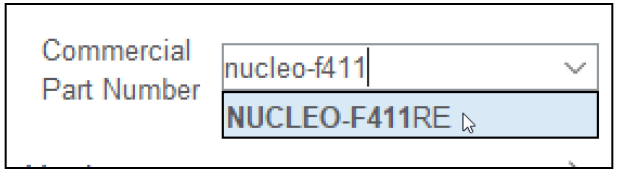


STM32 Nucleo-64 Evaluation board
STM32F411

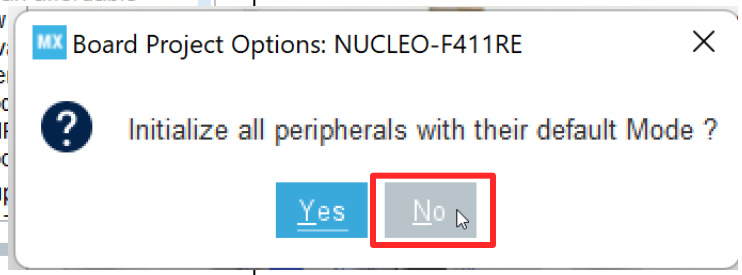
Start a new project in CubeMX

- NOTE: This following example is for "nucleo-f411"
- In CubeMX select "Start project from ST board"
- Type "nucleo-f411" to find Nucleo board
- Click "Start project" to start project
- This time select *No* to initialize peripherals



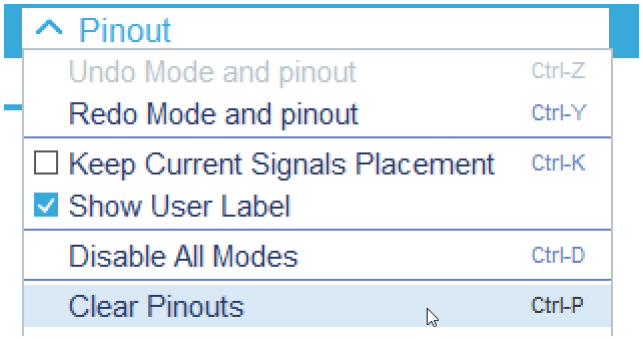


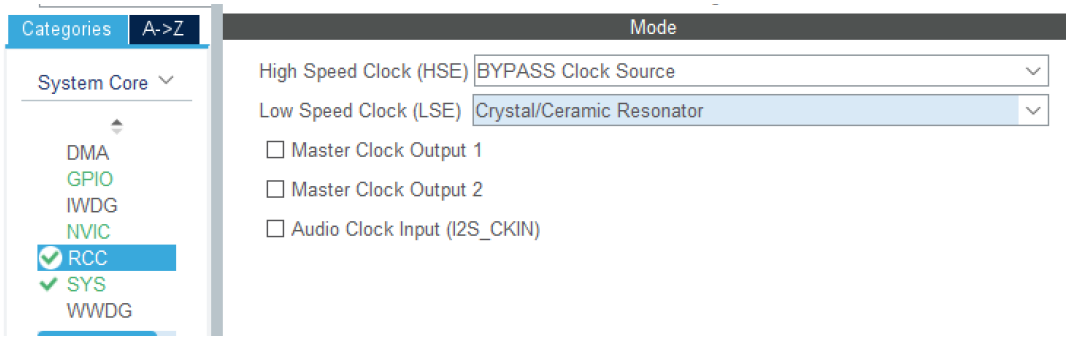


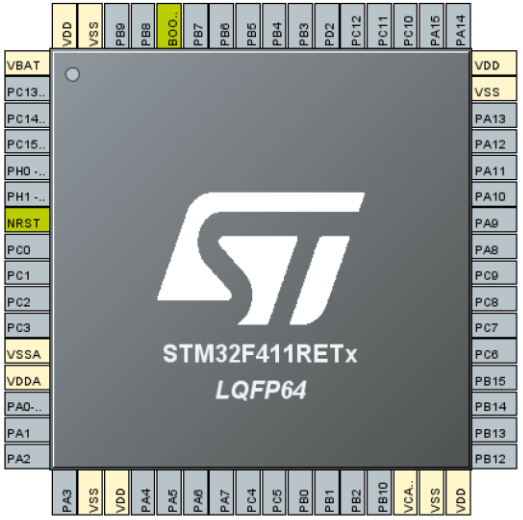


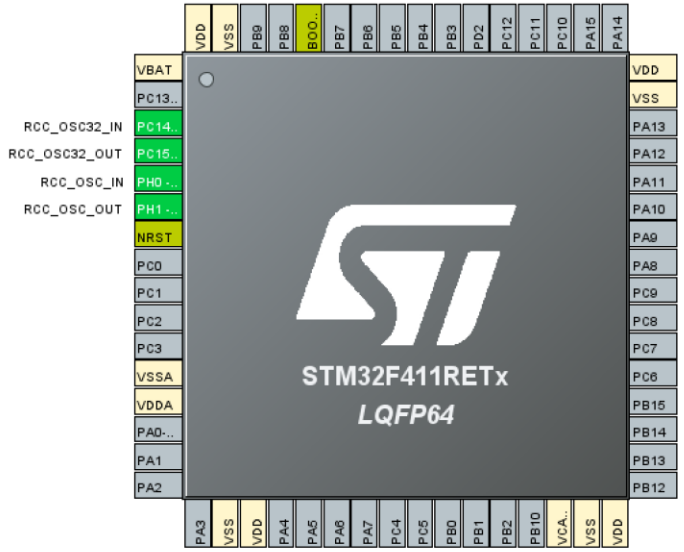
RCC setup

- Go to Pinout and “Clear Pinouts”
- *Go to Pinout and Configuration to set up the RCC clocks*



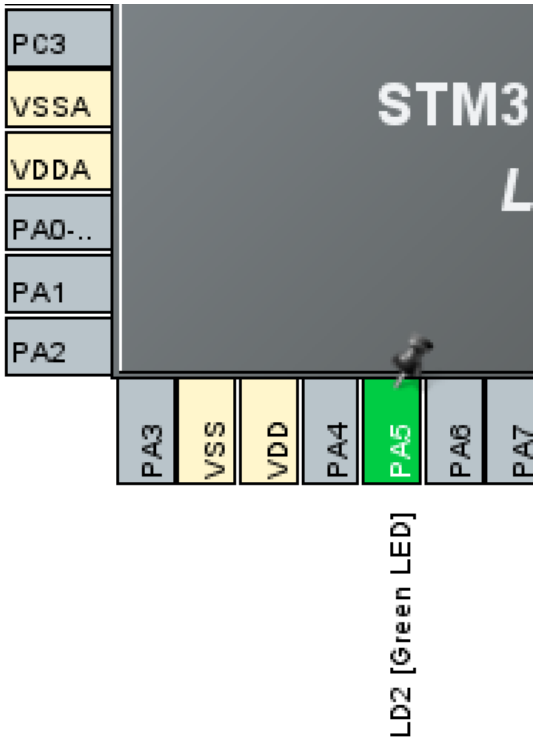


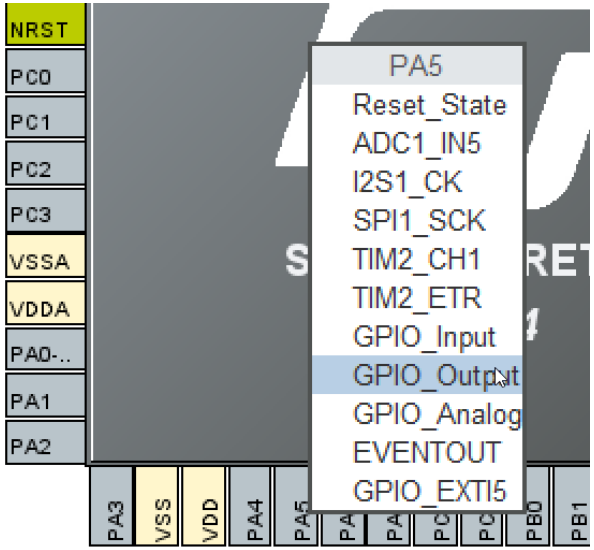


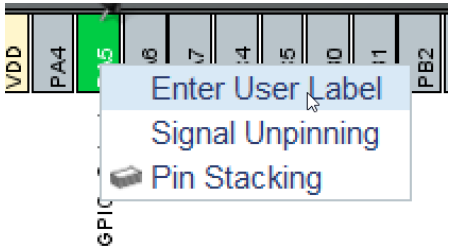


Green LED GPIO setup

- From documentation we know the green LED is connected to PA5
- Click PA5 in the GPIO/MCU diagram. Select GPIO output
- Right-click to enter user label. Call it LD2 [Green LED]
- When done it looks like this:

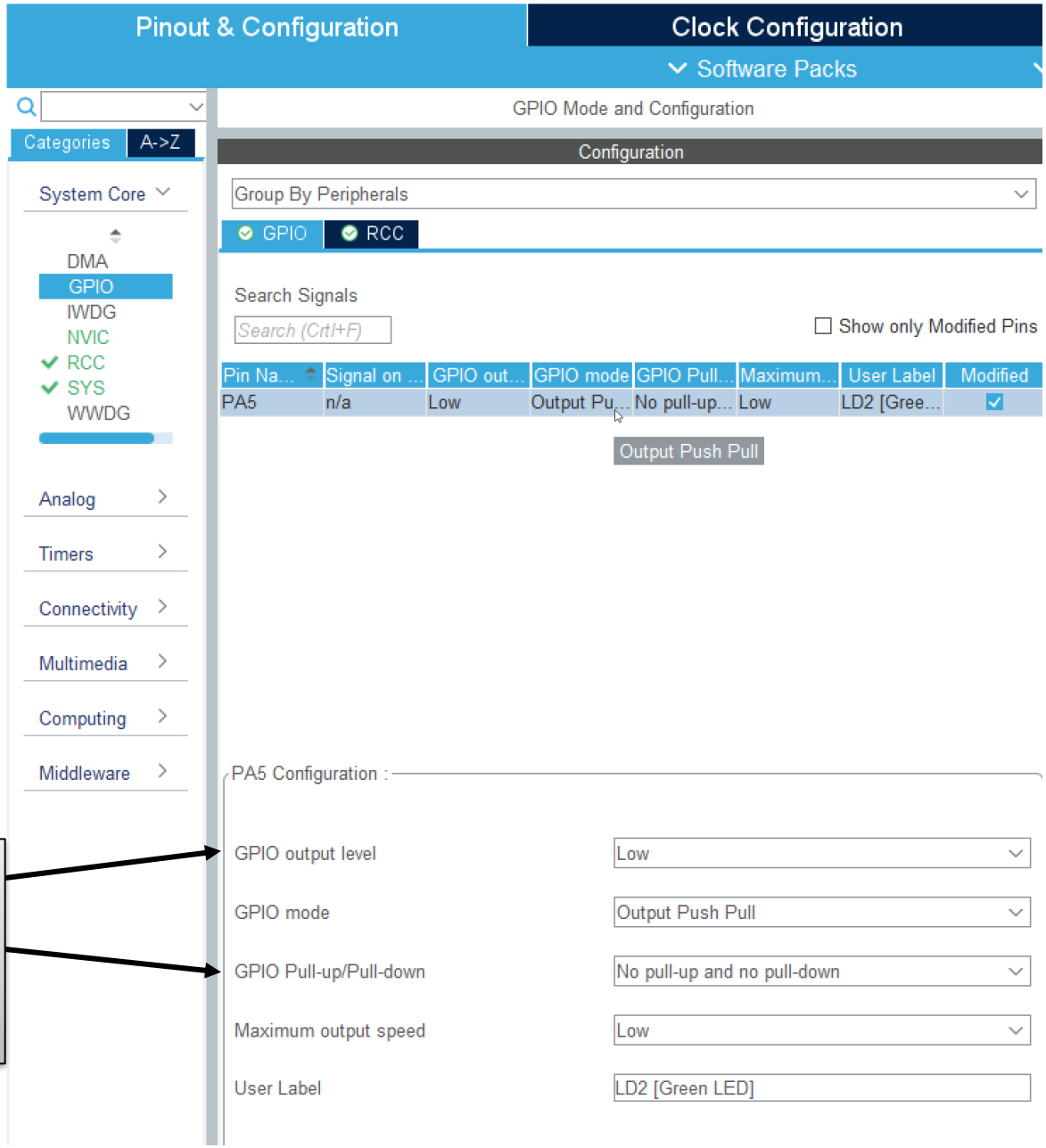






Checking GPIO configuration

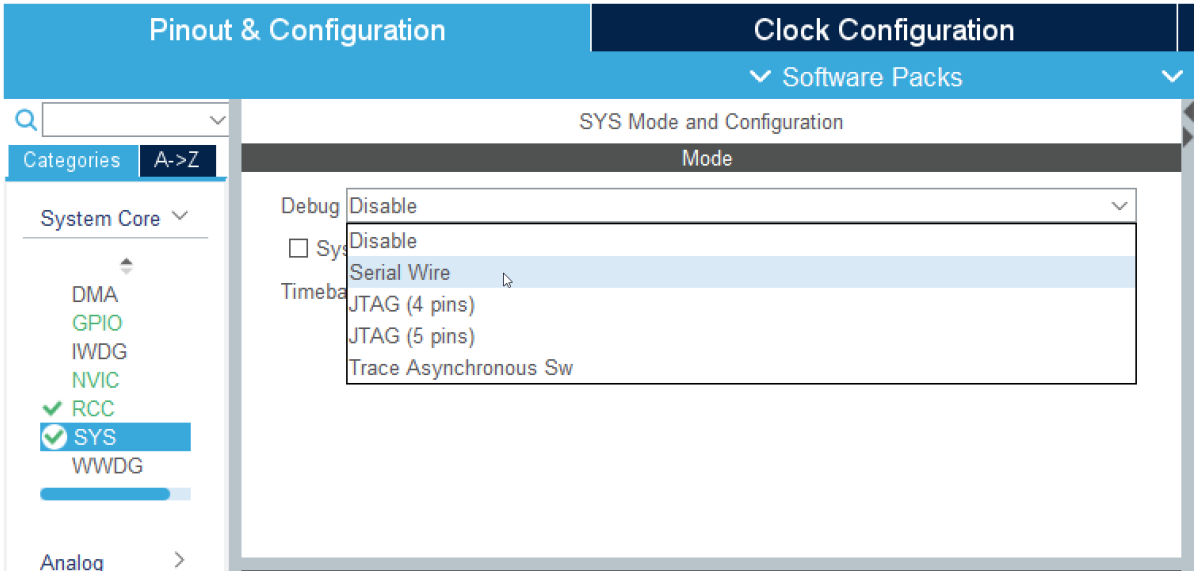
- Clicking GPIO will show a list of the activated GPIOs
- Clicking e.g. PA5 will give access to *GPIO Output* relevant settings (Push-pull operation, pull up resistors, etc..)
- Leave settings as is

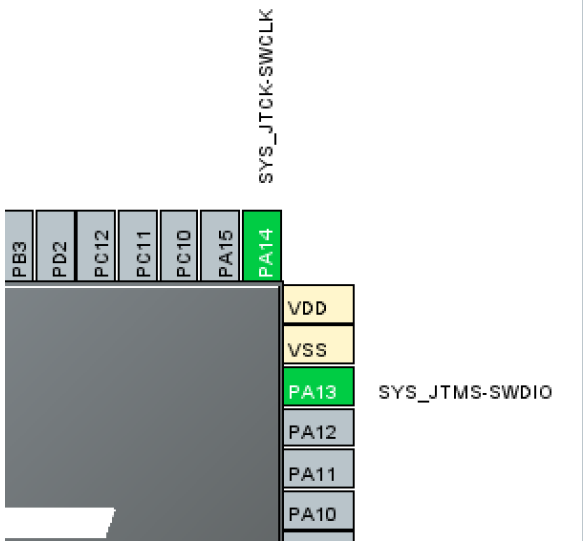


GPIO output level: after initialization, the pin will state in logic level high or low(output voltage 3.3V or 0V)
GPIO Pull-up/Pull-down: the pin is internally pulled up to the VCC line (3.3V), pulled down (GND), or floating.

Activating and selecting Debugger

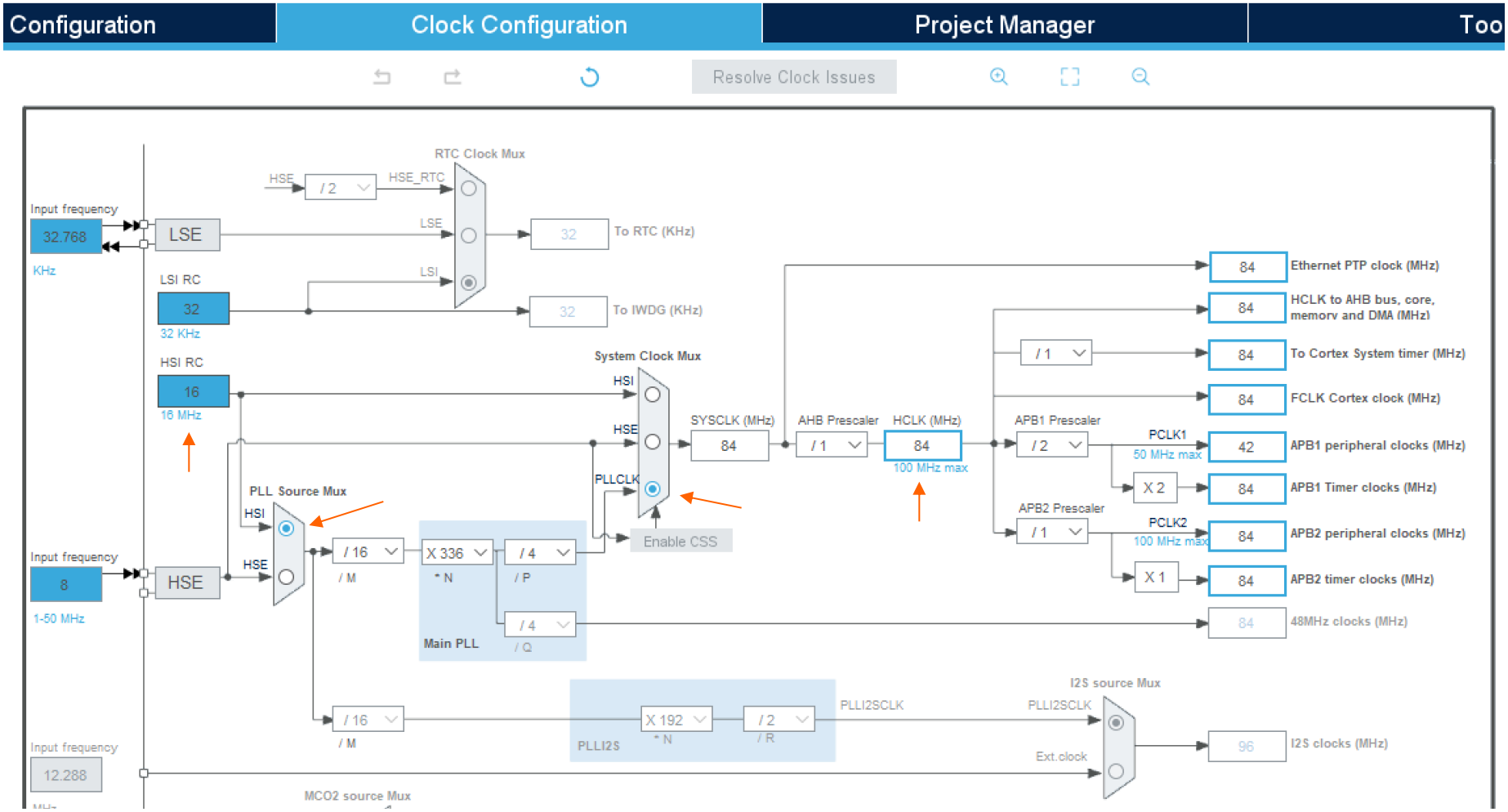
- Open *SYS* and select Debug to be “Serial Wire”.
  This will enable debugging options through the ST-LINK
- Notice how this enables additional pins in the *Pinout View*





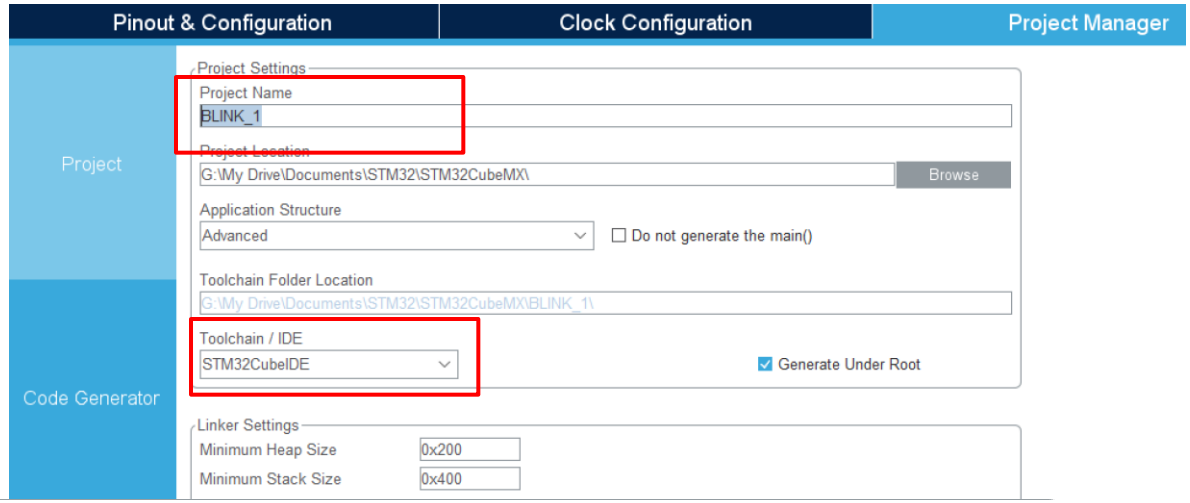
Clock configuration tab

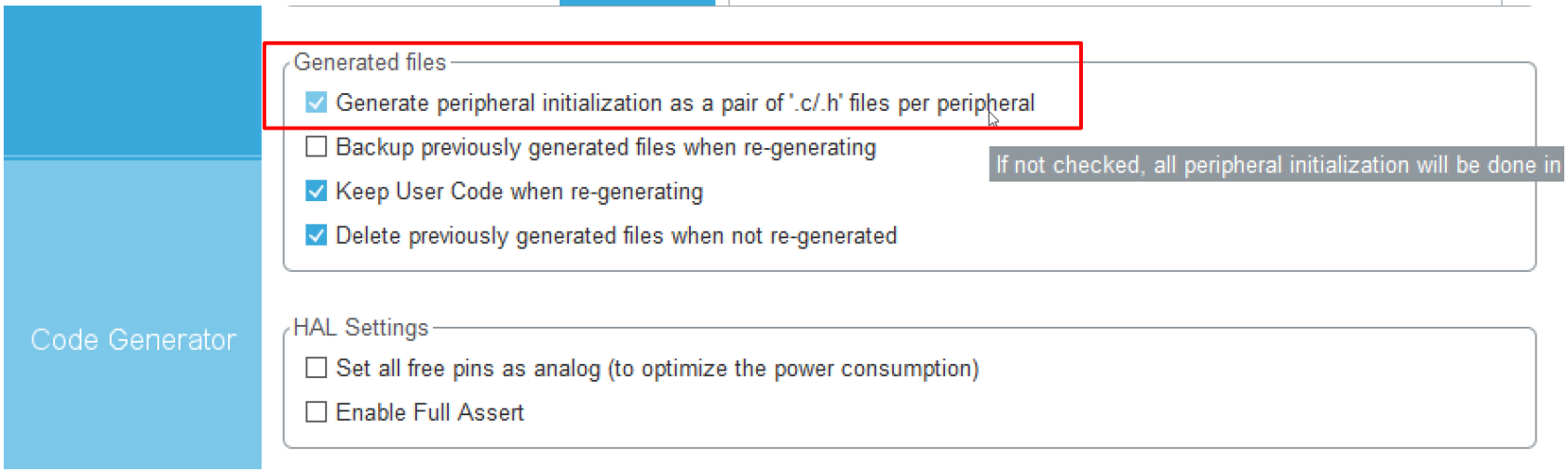
- Clock configuration we can leave as is (default) for now



Project Manager

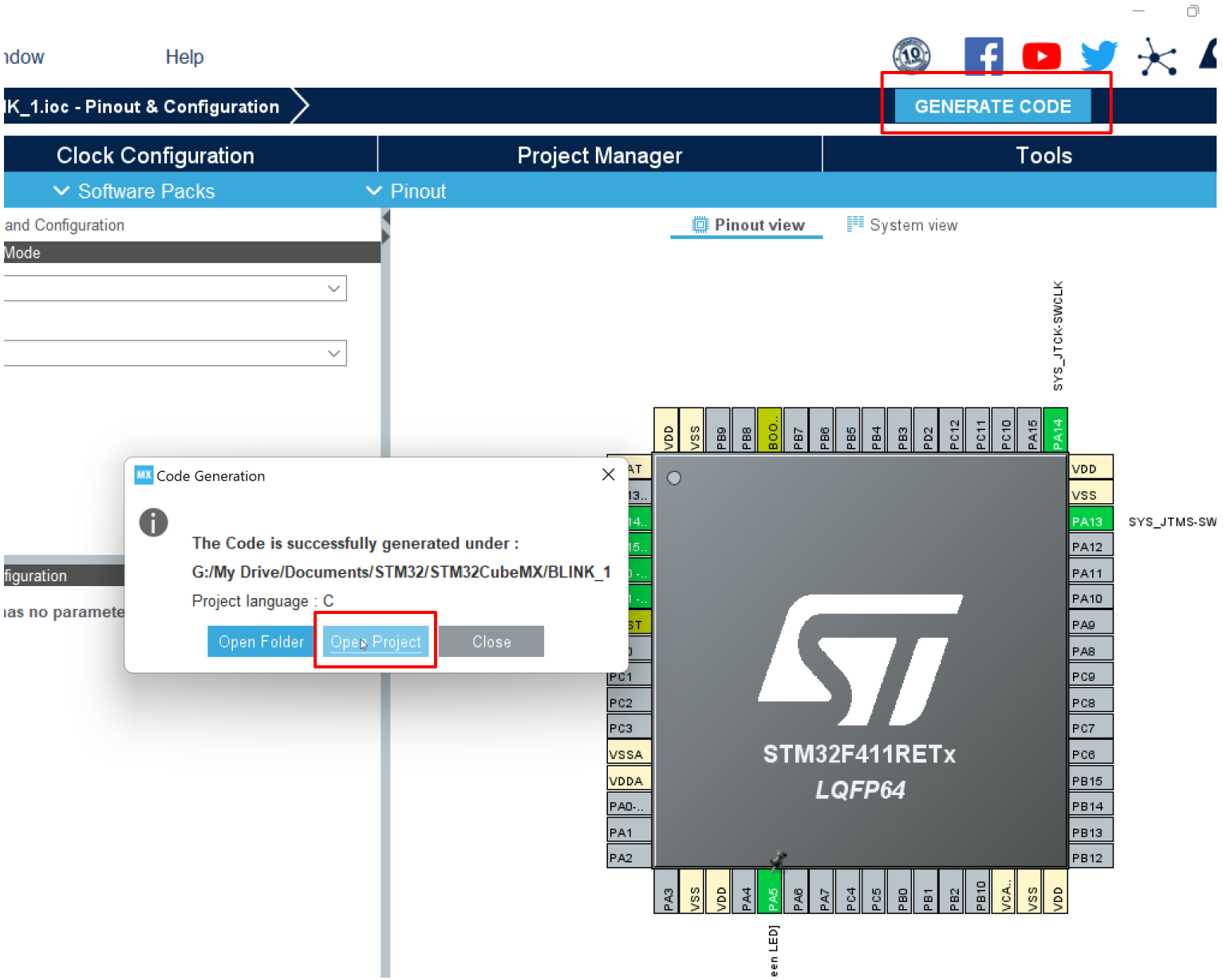
- Give the project a name, e.g. BLINK_1
- Select Toolchain to be *STM32CUBEIDE*
- In *Code Generator* mark *"Generate peripheral initialization as a pair of .c/.h …"*



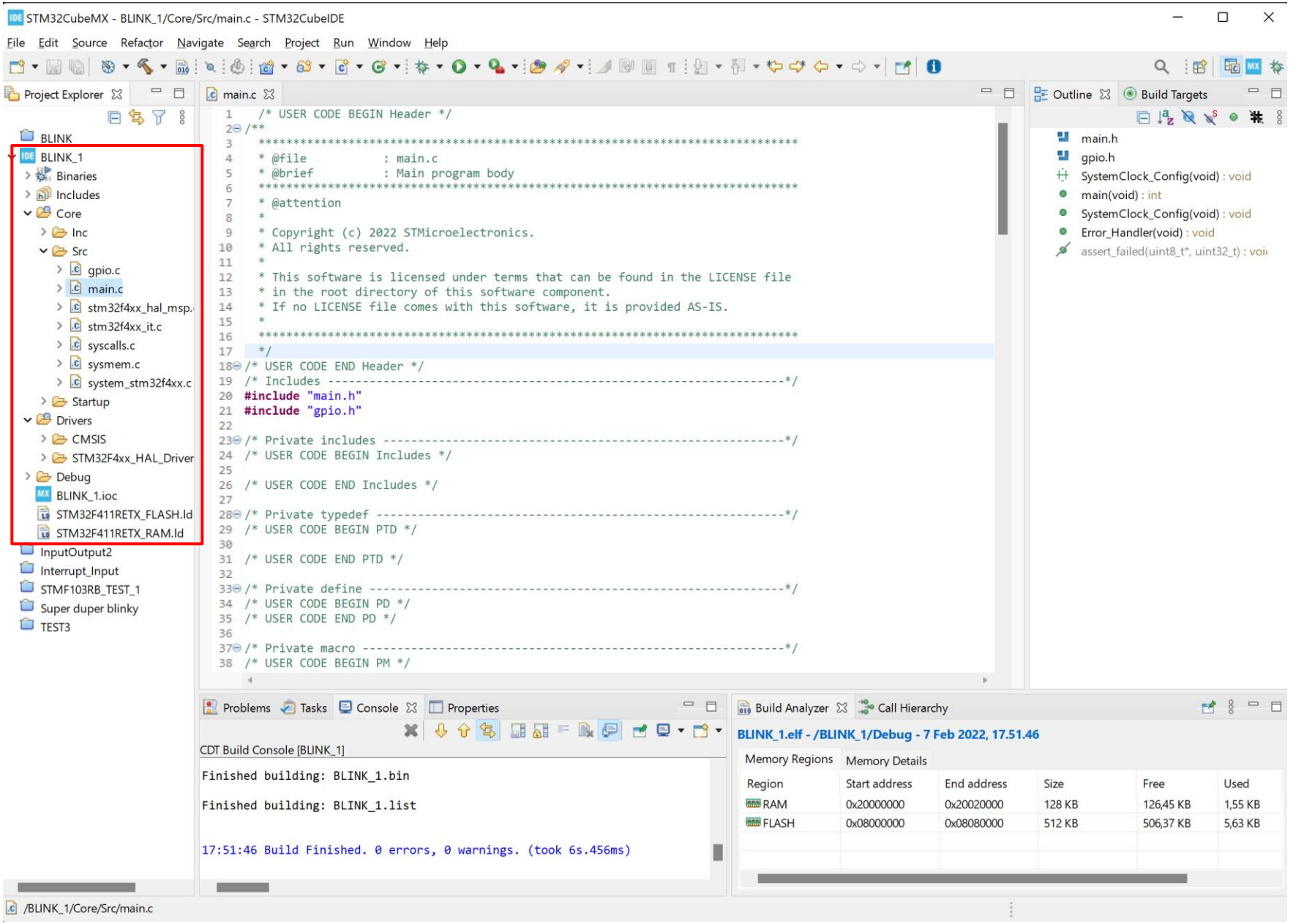


Generate code

- Now clock “Generate Code” and “Open Project”

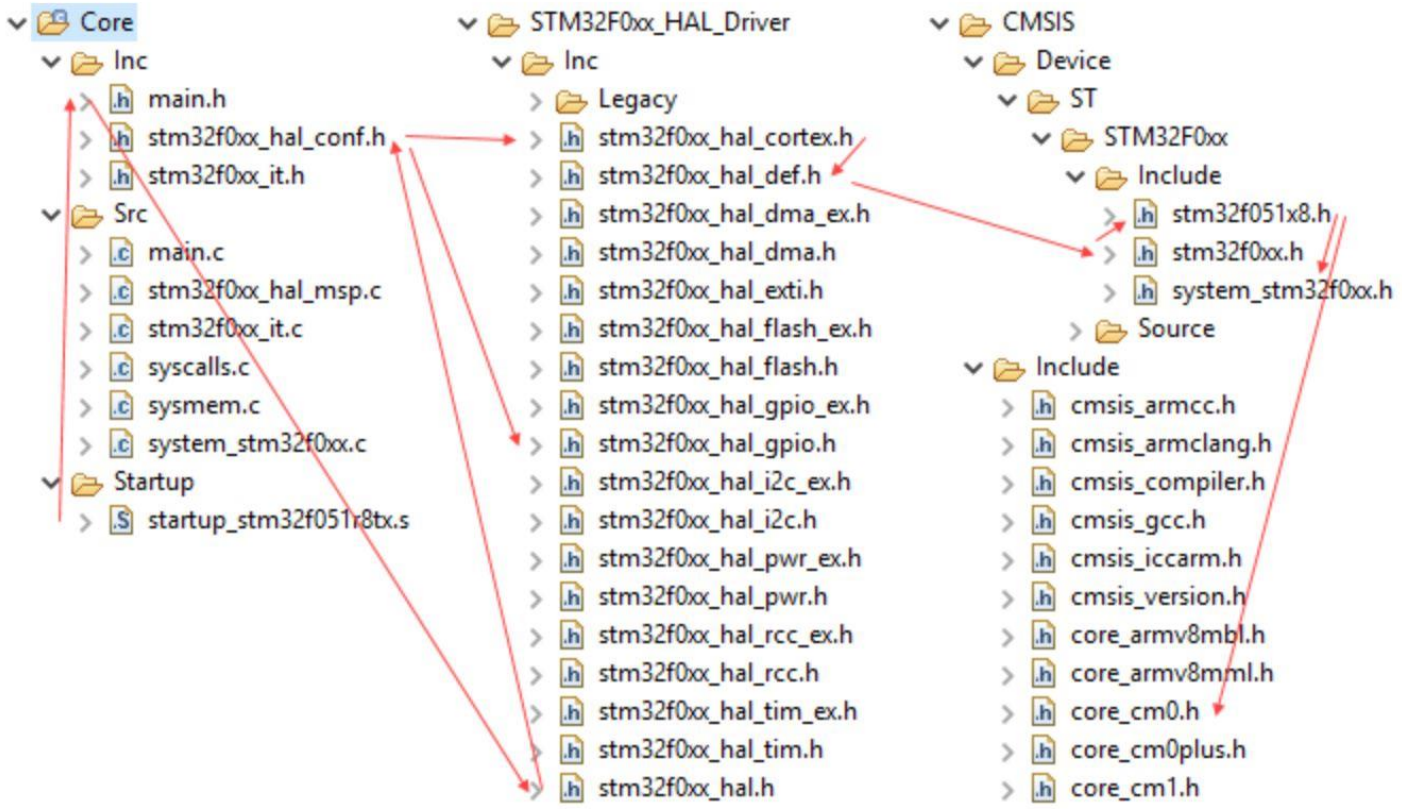


CubeIDE: In Core -> Src -> main.c is our main file



Code depencency / code structure

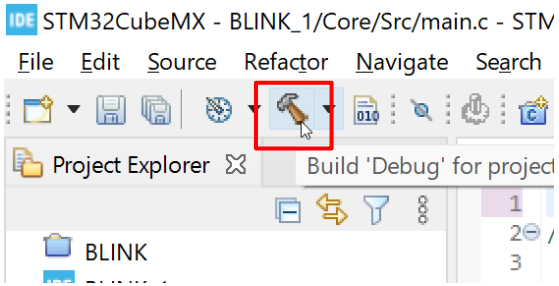
Code dependency starts from the `main.h` source file. This file includes HAL files which eventually includes CMSIS files. The `main` function is called from the startup file `startup_*.s`.



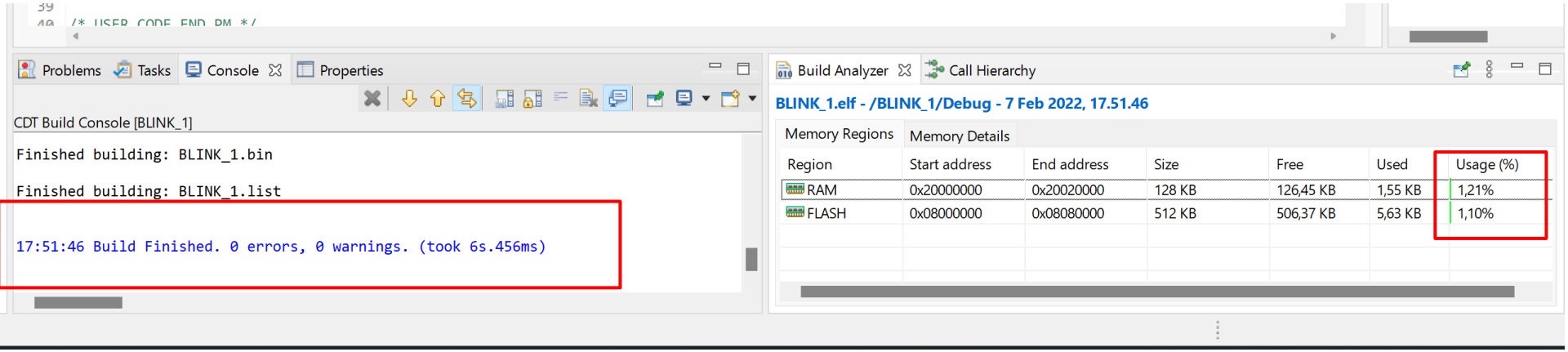
CMSIS: Common Microcontroller Software Interface Standard (CMSIS)

Source: https://www.codeinsideout.com/blog/stm32/intro/

Build, Download (to MCU) **and *Run***



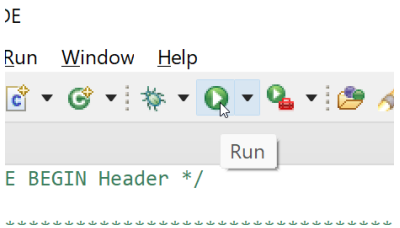
- Click the "hammer symbol" to build the project from our source code
- Building creates a binary (.bin) executable file that can be downloaded to the MCU
- Notice RAM and Flash usage



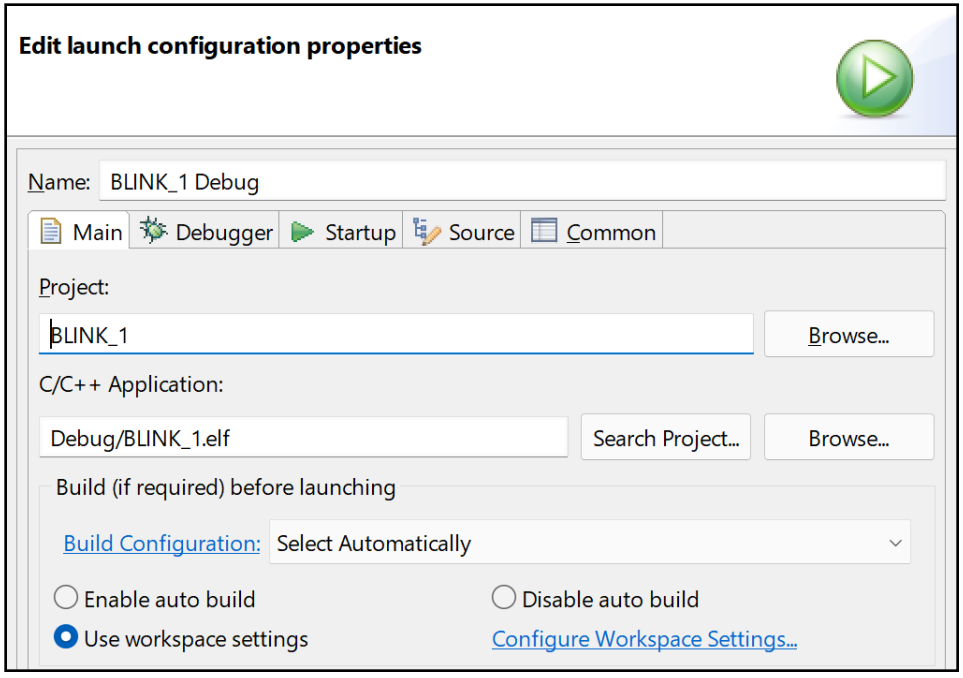
Build
In C, the Build Process is the process of converting the high-level source code representation of your embedded software into an executable binary image. Also called **compiling** and **linking**.

Download and Run

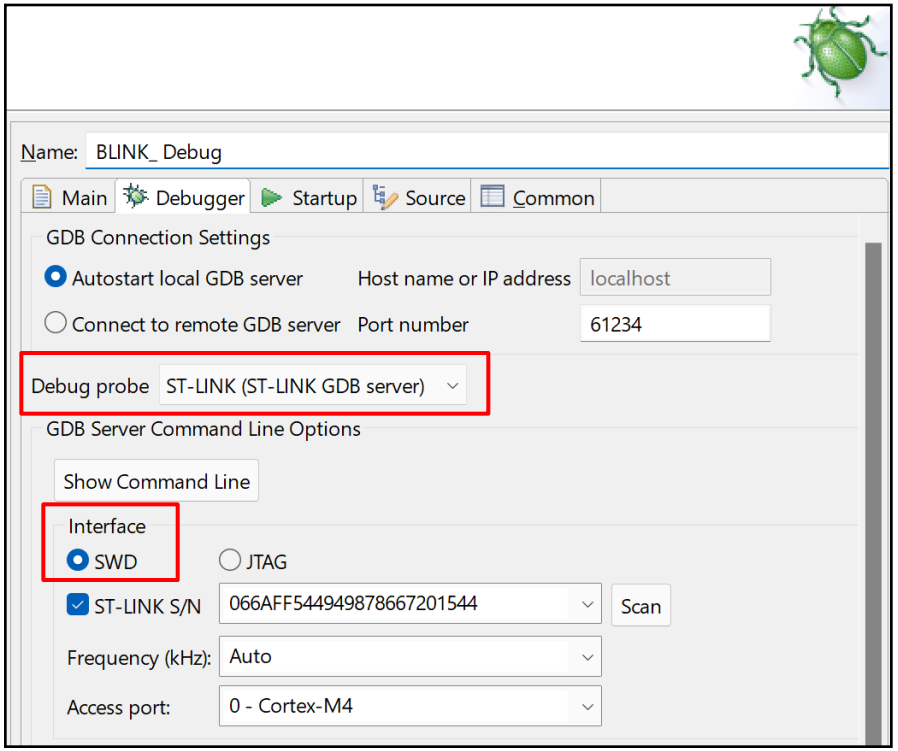
- Connect your Nucleo board to the PC, through the USB-mini port/cable
- Click the "play" symbol to start the Download (to MCU) and run process
- In the *Main* tab, check that the fields are populated like below
- In the *Debugger* tab, make sure *Debug probe* is set to *ST-LINK,* and *SWD* is checked.
- You may be prompted to update ST-LINK firmware.
- When ready, click OK to start download to board.



Main tab



Debugger tab



Adding functionality to the program code

- Locate **`while(1) {}`** about halfway into the main.c code
- This is typically where you would add user specific code
- Use **`HAL_GPIO_TogglePin`** function to make green LED blink every ½sec.
- Consult HAL manual (UM1725) for how to use **`HAL_GPIO_TogglePin`**
- On page 415 we find:

HAL_GPIO_TogglePin

Function name

void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)

Function description

Toggles the specified GPIO pins.

Parameters

- **GPIOx:** Where x can be (A..K) to select the GPIO peripheral for STM32F429X device or x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
- **GPIO_Pin:** Specifies the pins to be toggled.

Return values

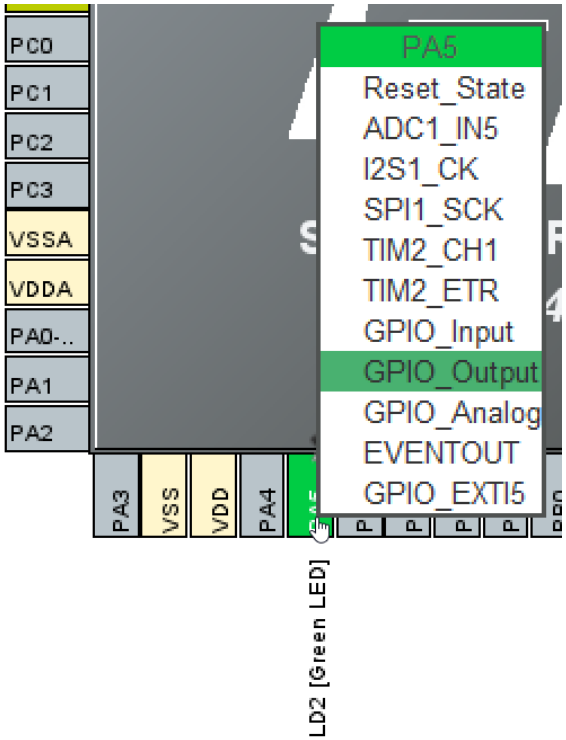
- **None:**

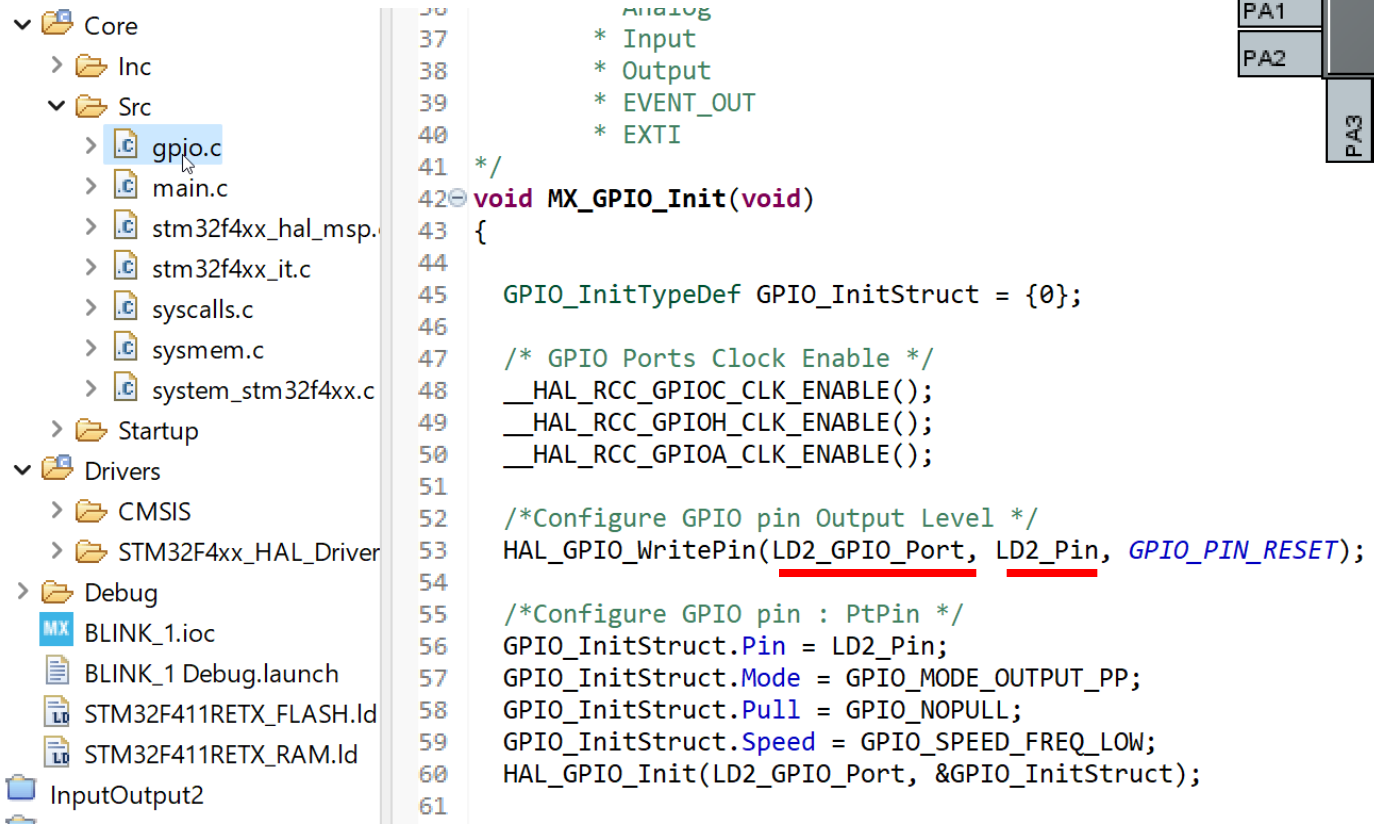
- So now we need to find (or remember) which GPIO port and GPIO pin green LED had.

Adding functionality to the program code

- From CubeMX we know, or see, that LD2 is on PA5. Port A, pin 5.

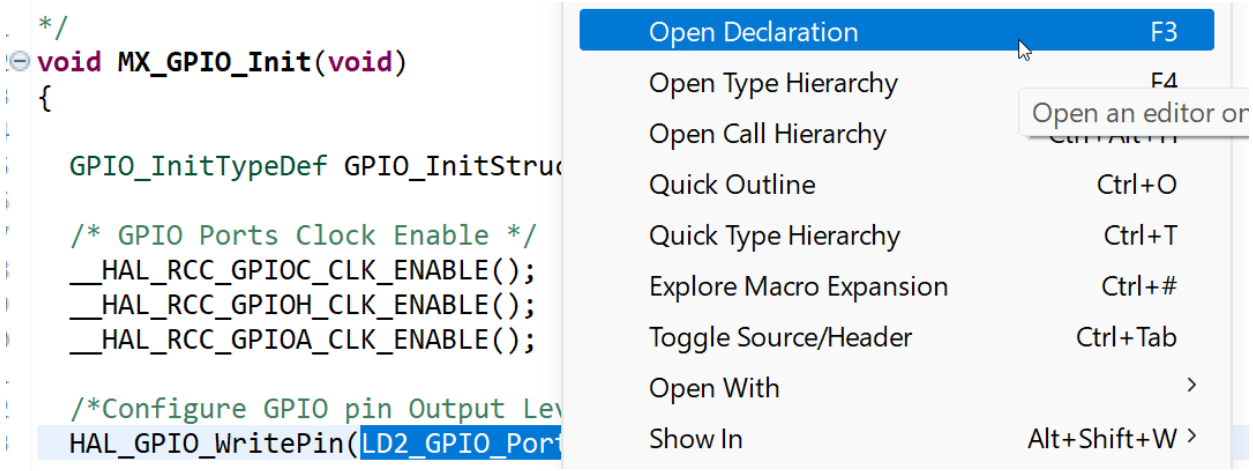
- To find correct naming we can look in *Src –> gpio.c* where GPIOs are (auto)configured by HAL





Adding functionality to the program code

- If we want to know more about how LD2_GPIO_Port and LD2_Pin are defined, right click on the variable name and select "Open Declaration".



- This will open the *main.h* header file which is where CubeMX has placed these definitions. It's seen that LD2_GPIO_Port is defined as **GPIOA** and LD2_Pin is defined as **GPIO_PIN_5**

```
58
59  /* Private defines -----------------------------------------------------------
60  #define LD2_Pin GPIO_PIN_5
61  #define LD2_GPIO_Port GPIOA
62  /* USER CODE BEGIN Private defines */
63
```

Put your code between the CODE BEGIN and CODE END tags. It retains the code in case of rebuild in CubeMX

Exercise 1: Adding functionality to the program code.

Use the Nucleo F411 example re-do for your F302 board.

- Now we can add the "blink functionality" to the program code:

```
93   while (1)
94   {
95     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
96     HAL_Delay(500);   /* Insert delay 500 ms */
97     /* USER CODE END WHILE */
98     /* USER CODE BEGIN 3 */
99   }
100  /* USER CODE END 3 */
```

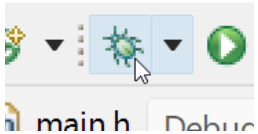
EX1: Look up the HAL_Delay() function in the HAL manual and explain what it does

EX2: Use the defined names found in gpio.c instead, and change frequency to 100ms.

EX3: Look in HAL manual for another way to manipulate the GPIO (set/reset) and use this to make LED blink.

EX4: Create a variable X (*char, int or uint8_t* type) that counts up from 0. With each step the LED should blink X times (speed 100ms). Incorporate a pause of 1 sec between each increase of X. 0, 1, 2, 3, 4 etc blinks..

EX5: Use the Debug function to monitor and observe value of X and steps through code. Make screenshots.



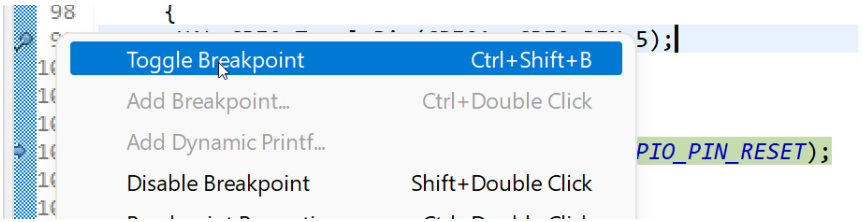
EX6: Use CubeMX to configure one more GPIO to make an externally added LED blink.



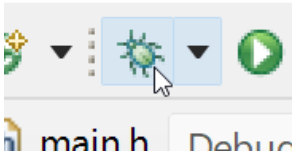
Upload code, screenshots from CubeMX and images of LED.

Using the debugging function

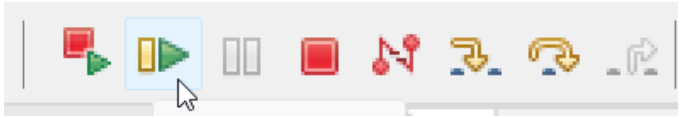
- Add any **breakpoints** you may want to add by right-clicking on the line-number where you want the break.
  Or, double-click in the blue bar where you want the breakpoint.



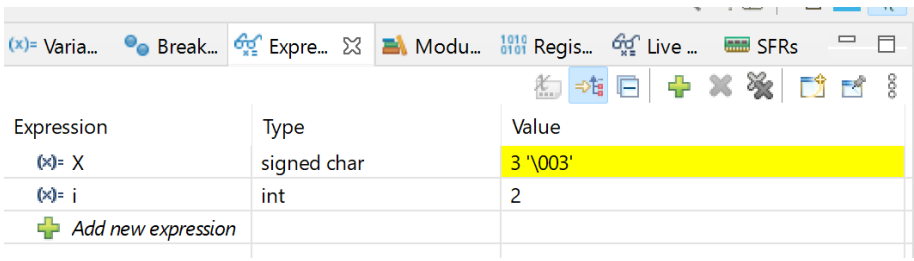
- When ready, click the "bug" symbol to start debugging. A new perspective opens.



- Debug **tools**: *Resume, Pause, Step Into, Step Over: Resume* will stop at first breakpoint.



- In the Debug monitor you can monitor variables, registers, etc.
  Click "Add new" to add known variables.
  Only *global variables* can be found/seen at all times.



Adding external components

- Refer to the User Manual (UM1724) and look for pinout diagrams. On page 32 we find this:

- Choose one. E.g. *PB13* and configure it as GPIO_Output in CubeMX. (PB13 = Morpho connector 30)

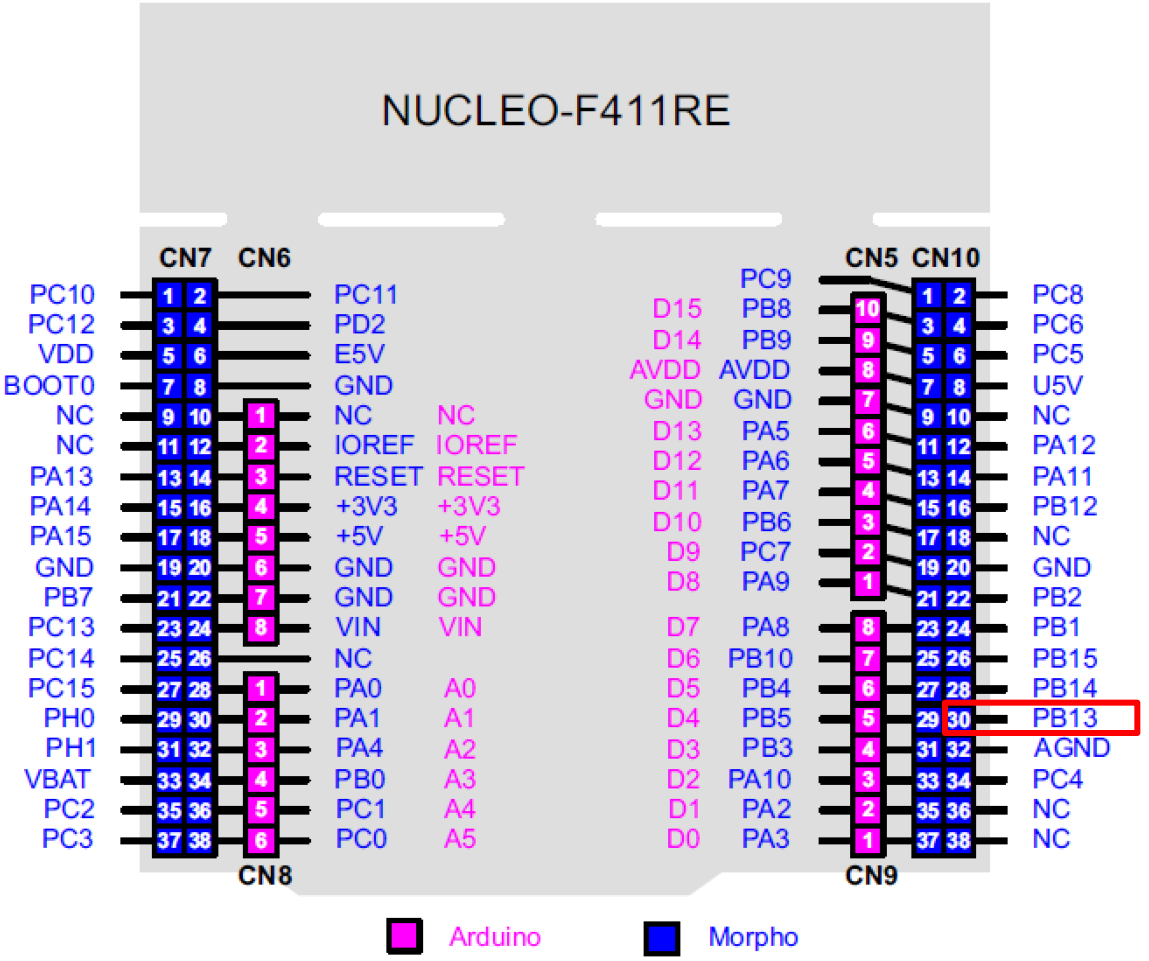
- Label the GPIO, e.g. exLED

- Use “Save as” to save and generate new code. Name it e.g. BLINK_2

- In CubeIDE the new GPIO be manipulated as the others

- Connect an LED and test it

Figure 19. NUCLEO-F411RE



Using STM32CubeMonitor to monitor variables & system

- CubeMonitor can be a valuable tool for monitoring & bug fixing.
- Download and install: https://www.st.com/en/development-tools/stm32cubemonitor.html
- Basic Flow example*:*
  myVariables, myPRobe_Out and *myProbe_In* needs configuring, as minimum

